

JavaScript Interview Questions

Array Methods — Concepts, Code & Answers

This guide covers the most commonly asked interview questions on JavaScript array methods — from basics like **map**, **filter**, **reduce** to trickier topics like mutating vs. non-mutating methods, **flat/flatMap**, and polyfills. Each question includes a clear explanation and a short code example.

Contents

1. Core Concepts
2. Iteration Methods (forEach, map, filter, reduce)
3. Search & Check Methods (find, includes, some, every)
4. Mutating vs. Non-Mutating Methods
5. Advanced & Tricky Questions
6. Coding Challenges

1. Core Concepts

Q1. What is the difference between a mutating and a non-mutating array method?

Mutating methods change the original array in place and often return something other than a new array (e.g. the new length or the removed element). Non-mutating methods leave the original array untouched and return a new array or value instead.

EXAMPLE

```
const arr = [3, 1, 2];

// Mutating
arr.push(4);           // arr is now [3, 1, 2, 4]
arr.sort();            // arr is now [1, 2, 3, 4]

// Non-mutating
const doubled = arr.map(x => x * 2); // new array, arr unchanged
```

Q2. How do you check if a variable is really an array?

Use `Array.isArray(value)`. Checking `typeof value === 'object'` is not reliable because arrays and plain objects both return "object".

```
Array.isArray([1, 2, 3]); // true
Array.isArray('hello');   // false
```

Q3. What's the difference between `slice()` and `splice()`?

`slice(start, end)` is non-mutating and returns a shallow copy of a portion of the array. `splice(start, deleteCount, ...items)` is mutating — it removes and/or inserts elements directly in the original array.

```
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 3);           // [2, 3] -- arr unchanged
arr.splice(1, 2, 'a');     // removes [2,3], inserts 'a'
// arr is now [1, 'a', 4, 5]
```

2. Iteration Methods

Q4. What is the difference between `forEach()` and `map()`?

`forEach` executes a callback for each element and always returns undefined — it's used purely for side effects. `map` also runs a callback on each element but returns a **new array** built from the callback's return values.

```
const nums = [1, 2, 3];

nums.forEach(n => console.log(n * 2)); // logs 2,4,6 -> returns undefined
const doubled = nums.map(n => n * 2);  // returns [2, 4, 6]
```

Q5. Explain `reduce()` and give a use case beyond summing numbers.

`reduce(callback, initialValue)` runs a reducer function over each element, accumulating a single result. Besides summing, it's commonly used to group data, flatten arrays, or build lookup objects.

```
const people = [{name:'Amy', dept:'HR'}, {name:'Sam', dept:'Eng'},
                 {name:'Lee', dept:'HR'}];

const byDept = people.reduce((acc, p) => {
  (acc[p.dept] ??= []).push(p.name);
  return acc;
}, {});
// { HR: ['Amy', 'Lee'], Eng: ['Sam'] }
```

Q6. Can you break out of a `forEach()` loop early?

No. `forEach` has no built-in way to stop mid-iteration — `break`, `return`, and `continue` don't work as expected inside it. If early exit is needed, use a plain `for` loop, `for...of`, or `some()`/`every()`, which do stop early.

```
// Stops as soon as callback returns true
[1, 2, 3, 4].some(n => {
  if (n === 3) return true; // "breaks" the loop
  console.log(n);
  return false;
});
```

3. Search & Check Methods

Q7. What's the difference between find(), filter(), and indexOf()?

find returns the **first matching element** (or undefined). filter returns **all matching elements** as a new array. indexOf searches by strict equality and returns an **index** (or -1), not by a test function.

```
const nums = [5, 12, 8, 130, 44];
nums.find(n => n > 10);    // 12  (first match)
nums.filter(n => n > 10);  // [12, 130, 44]
nums.indexOf(8);          // 2
```

Q8. What's the difference between includes() and indexOf() for checking existence?

Both check membership, but includes() returns a boolean and correctly detects NaN using SameValueZero comparison, while indexOf() uses strict equality and can never find NaN (since NaN !== NaN).

```
[1, NaN, 3].indexOf(NaN);  // -1  (never found)
[1, NaN, 3].includes(NaN); // true
```

Q9. Explain some() and every() with an example.

some() returns true if **at least one** element passes the test; every() returns true only if **all** elements pass. Both short-circuit as soon as the result is known.

```
const ages = [18, 22, 16, 30];
ages.some(a => a < 18);  // true  (at least one minor)
ages.every(a => a >= 18); // false (not everyone is an adult)
```

4. Mutating vs. Non-Mutating Methods

Q10. List common mutating methods vs. non-mutating methods.

Mutating (changes original)	Non-mutating (returns new)
push, pop, shift, unshift	map, filter, slice
splice, sort, reverse	concat, flat, flatMap
fill, copyWithin	reduce, join, toSorted*

*ES2023 introduced non-mutating counterparts: toSorted(), toReversed(), toSpliced(), and with().

Q11. Why is sort() considered a common source of bugs?

By default, sort() converts elements to strings and sorts lexicographically — so numeric arrays sort incorrectly unless you supply a compare function. It's also mutating, which surprises developers expecting a new array.

```
[10, 1, 21, 2].sort();
// ['1', '10', '2', '21']  -- wrong! sorted as strings

[10, 1, 21, 2].sort((a, b) => a - b);
```

```
// [1, 2, 10, 21] -- correct numeric sort
```

5. Advanced & Tricky Questions

Q12. What do flat() and flatMap() do?

`flat(depth)` flattens nested arrays up to the given depth (default 1). `flatMap()` is equivalent to calling `map()` followed by `flat(1)`, but is more efficient since it does it in a single pass.

```
[1, [2, 3], [4, [5]]].flat();      // [1, 2, 3, 4, [5]]
[1, [2, 3], [4, [5]]].flat(2);    // [1, 2, 3, 4, 5]

[1, 2, 3].flatMap(n => [n, n * 2]);
// [1, 2, 2, 4, 3, 6]
```

Q13. Why does `array.length = 0` empty an array, and are there other ways?

Array length is a writable property tied to the array's internal storage; setting it to 0 truncates the array immediately, mutating the original reference (unlike `arr = []`, which just points the variable at a new array).

```
let arr = [1, 2, 3];
const ref = arr;

arr.length = 0;  // ref is also [] -- same array object
arr = [];        // ref would still be [1,2,3] if reassigned this way
```

Q14. How would you remove duplicate values from an array?

The most common approach uses a `Set`, which only stores unique values, then spreads it back into an array.

```
const nums = [1, 2, 2, 3, 4, 4, 5];
const unique = [...new Set(nums)];
// [1, 2, 3, 4, 5]
```

Q15. What happens if you use an array method's callback with the wrong 'this'?

Methods like `map`, `filter`, and `forEach` accept an optional second argument to set `this` inside a regular function callback. Arrow functions ignore this entirely since they capture `this` lexically — a frequent point of confusion in interviews.

```
const ctx = { factor: 2 };
[1, 2, 3].map(function (n) {
  return n * this.factor;  // needs thisArg
}, ctx);
// [2, 4, 6]
```

Q16. How is a sparse array handled differently by `forEach/map` vs. a `for` loop?

Methods like `forEach`, `map`, and `filter` skip empty slots in sparse arrays entirely — the callback is never invoked for them. A plain `for` loop, by contrast, iterates every index and returns `undefined` for empty slots.

```
const sparse = [1, , 3]; // empty slot at index 1
sparse.map(x => x * 2);   // [2, <1 empty item>, 6]
sparse.forEach(x => console.log(x)); // logs 1, 3 (skips the hole)
```

6. Coding Challenges

Q17. Implement your own version of `Array.prototype.map` (a polyfill).

A classic interview task to test understanding of how array methods work under the hood:

```
Array.prototype.myMap = function (callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) {
      result[i] = callback.call(thisArg, this[i], i, this);
    }
  }
  return result;
};

[1, 2, 3].myMap(n => n * 2); // [2, 4, 6]
```

Q18. Given an array of numbers, find the two elements that sum to a target value.

The efficient $O(n)$ solution uses a hash map (or Set) to track values already seen, avoiding the $O(n^2)$ brute-force double loop.

```
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (seen.has(complement)) return [seen.get(complement), i];
    seen.set(nums[i], i);
  }
  return null;
}

twoSum([2, 7, 11, 15], 9); // [0, 1]
```

Q19. Group an array of objects by a property using `reduce()`.

```
const orders = [
  { id: 1, status: 'done' }, { id: 2, status: 'pending' },
  { id: 3, status: 'done' }
];

const grouped = orders.reduce((acc, order) => {
  (acc[order.status] ??= []).push(order);
  return acc;
}, {});
// { done: [...], pending: [...] }
```

Q20. Chain array methods to find the total price of items in stock, over \$10.

A good example of composing filter, map, and reduce together — a very common "show me how you think" interview question.

```
const items = [
  { name: 'Pen', price: 2, inStock: true },
```

```
{ name: 'Desk', price: 150, inStock: true },  
{ name: 'Chair', price: 80, inStock: false },  
{ name: 'Lamp', price: 25, inStock: true },  
];  
  
const total = items  
  .filter(item => item.inStock && item.price > 10)  
  .map(item => item.price)  
  .reduce((sum, price) => sum + price, 0);  
// total = 175 (Desk + Lamp)
```

Tip for interviews: whenever you reach for an array method, be ready to say whether it mutates the original array, what it returns, and its time complexity — interviewers often probe exactly these three points.